《算法设计与分析》实验报告

实验六　概率算法

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	2025/06/03
班级	22 级计算机 2 班		

实验内容

[本次实验无 OJ 测试]

本实验无需扩展部分

1）设计一个洗牌程序，要求不同于课本例子的方案。
2）试用拉斯维加斯方法求解 8 皇后问题，并说明拉斯维加斯方法的特点。
3）举例并实现一个舍伍德方法的应用及特点。
4）用蒙特卡洛方法，解决 GPS 在隧道中无信号时的导航问题。

实验目的

1）掌握舍伍德、拉斯维加斯、蒙特卡洛三个随机算法。

2）学习掌握本类算法的设计与实现，完成实验报告书写。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

无 OJ 要求

实验过程和步骤

【1、实验题目：设计一个洗牌程序】

【2、程序代码及注释】

```
1. #include<stdio.h>
2. #include<string.h>
3. #include<time.h>
4. #include<stdlib.h>
```

```
5.
6. typedef struct card
7. {
8.     char suit[10];    //花色
9.     char face[10];    //牌面
10.} CARD;
11.
12.void Initialize(int result[]);     //按新牌顺序编号
13.void Shuffle(int result[]);        //通过对编号随机置乱，实现洗牌
14.void FillCard(CARD card[]);        //填充牌面和花色
15.void Deal(CARD card[], int result[]);  //输出
16.
17.void Initialize(int result[])
18.{
19.    int i;
20.    for(i=0;i<54;i++) // 修改为54张牌
21.    {
22.        result[i]=i;
23.    }
24.}
25.
26.void Shuffle(int result[]) {
27.    int n = 54; // 数组的长度
28.    int iterations = 1000; // 定义热力学洗牌的迭代次数，可根据需要调整
29.
30.    for (int outer = 0; outer < iterations; outer++) {
31.        for (int i = 0; i < n - 1; i++) {
32.            // 以一定的概率交换相邻元素
33.            if (rand() % 2 == 0) {  //在这里我们设置以50%的概率逐步随机地交换元素
34.                int temp = result[i];
35.                result[i] = result[i + 1];
36.                result[i + 1] = temp;
37.            }
38.        }
39.    }
40.}
41.
42.void FillCard(CARD card[])
43.{
44.    int i;
45.    char suits[5][10] = {"红心", "方块", "黑桃", "梅花", "小丑
```

```
    "}; // 添加Jokers 花色
    char faces[15][10] = {"2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A", "黑", "红"}; // 添加大小王牌面

    for(i = 0; i < 52; i++)
    {
        strcpy(card[i].suit, suits[i / 13]);
        strcpy(card[i].face, faces[i % 13]);
    }
    // 添加两张Joker 牌
    strcpy(card[52].suit, suits[4]); // Jokers 花色
    strcpy(card[52].face, faces[13]); // Small Joker 牌面
    strcpy(card[53].suit, suits[4]); // Jokers 花色
    strcpy(card[53].face, faces[14]); // Big Joker 牌面
}

void Deal(CARD card[], int result[])
{
    int i;
    for(i = 0; i < 54;) // 修改为54 张牌
    {
        printf("%s%s\n", card[result[i]].suit, card[result[i]].face);
        i++; // 确保循环正确递增
    }
}

int main()
{
    srand(time(NULL)); // 初始化随机数生成器
    CARD card[54]; // 修改为54 张牌
    int result[54]; // 修改为54 张牌
    FillCard(card);           //将牌面和花色顺序存放于结构体数组card
    Initialize(result);       //将牌编号存放于数组result 中
    Deal(card,result);       //输出洗牌前的结果
    Shuffle(result);          //洗牌
    printf("在洗牌后：\n");
    Deal(card,result);       //输出洗牌后的结果
    return 0;
}
```

【3、遇到的问题及解决方法】

当我在短时间多次运行程序，所得到的结果相同，这是因为 time 的精度为秒，因此短时

间多次运行会有相同的种子。

【4、算法分析与知识点】

热力学洗牌（Thermodynamic Shuffle）是一种基于物理学中热运动模型设计的洗牌方法。该算法的核心思想是模拟热力学系统中粒子的随机运动，通过不断扰动局部状态（即随机交换相邻两张牌），逐步使整个牌组趋于无序状态，从而达到“洗牌”的效果。

在具体实现中，热力学洗牌采用两层循环结构。外层循环控制扰动次数，即洗牌的迭代轮数；内层循环遍历整个牌组，对每一对相邻的牌以 50%的概率进行交换。这种方式通过大量的微小扰动逐步逼近理想的随机分布。

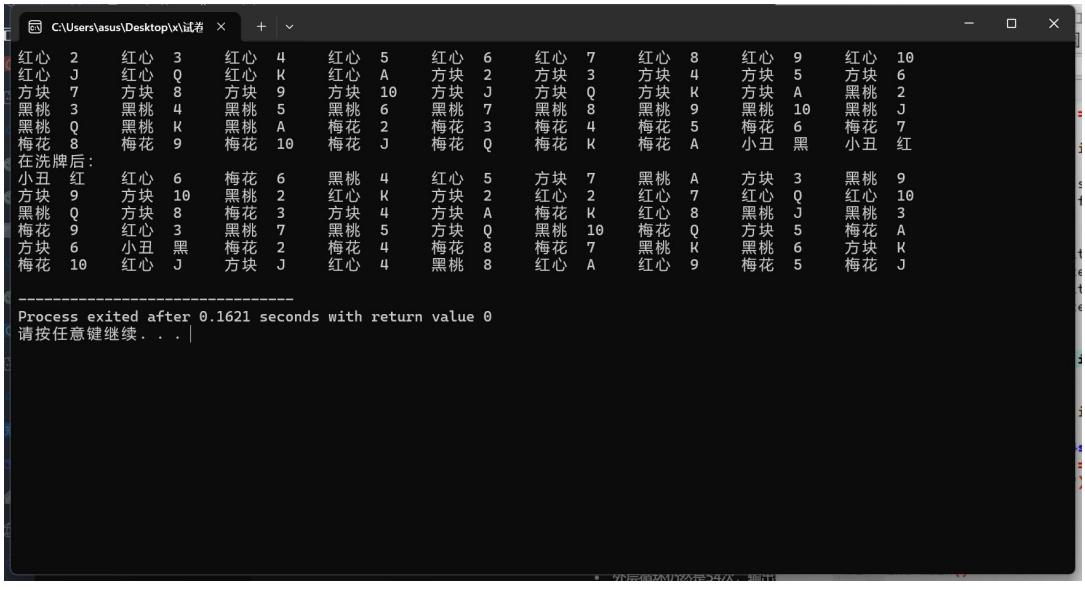
外层迭代次数是控制洗牌效果的关键参数。若次数过少，洗牌可能不够充分，牌序仍具有明显的初始顺序特征；若次数过多，则会增加不必要的计算开销。因此，在实际应用中需根据需求权衡随机性与计算效率。一般来说，将迭代次数设为几百到几千次能达到较好的随机性效果。本程序中设置为 1000 次，已能实现较好的洗牌混合度。

此外，该算法具有如下复杂度特性：

- **时间复杂度**：每次外层循环执行一次完整的内层循环，内层循环长度为 n−1n-1n−1，因此总时间复杂度为 $O(n \cdot k)$，其中 nnn 为牌数，kkk 为迭代次数。对于本程序中的 54 张牌和 1000 次迭代，理论执行次数为 $53 \times 1000 = 5300053 \times 1000 = 53000$，是线性规模内的可控复杂度。
- **空间复杂度**：该算法不需要额外的存储空间，仅在原数组 result[] 上进行就地交换，因此空间复杂度为 $O(1)$，即常数级。

综上所述，热力学洗牌算法结构简单、实现方便，适合对局部扰动具有良好容忍度的洗牌场景，但其在收敛到完全均匀分布方面效率略逊于经典的 Fisher-Yates 洗牌。若对随机性要求极高，可考虑将其与其他洗牌算法配合使用。

【5、AC 截图和运行测试截图】



【1、实验题目：拉斯维加斯方法求解 8 皇后问题】

【2、代码实现】

```
1. #include <iostream>
```

```cpp
#include <vector>
#include <random>
#include <ctime>
using namespace std;

// 定义一个生成随机数的类
class RandomNumber {
public:
    // 构造函数，如果不指定种子，就使用当前时间作为种子
    RandomNumber(unsigned long s = std::random_device{}()) {
        rng.seed(s);
    }

    // 生成范围在[0, n-1]之间的随机整数
    unsigned short Random(unsigned long n) {
        uniform_int_distribution<unsigned short> dist(0, n - 1);
        return dist(rng);
    }

private:
    mt19937 rng; // Mersenne Twister 随机数生成器
};

// 定义一个皇后类
class Queen {
    friend bool nQueen(int); // 声明友元函数用于解决 N 皇后问题

private:
    bool Place(int k);       // 检查在第 k 列放置皇后是否会导致冲突
    bool Backtrack(int t);   // 使用回溯算法解决从第 t 列开始的 N 皇后问题
    bool QueensLV(int stopVegas); // 使用拉斯维加斯算法解决 N 皇后问题
    int n;                   // 皇后数量
    vector<int> x, y;        // x[i]表示放置在第 i 列的皇后所位于的行号

    // 私有构造函数，初始化皇后数量和 x, y 向量
    Queen(int n) : n(n), x(n + 1), y(n + 1) {}
};


```

```
41.// 检查在第 k 列放置皇后是否会导致冲突
42.bool Queen::Place(int k) {
43.    for (int j = 1; j < k; ++j) { // 遍历前面已经放置的皇后
44.        // 判断是否在同一对角线上或同一行有皇后，如果有返回false
45.        if (abs(k - j) == abs(x[j] - x[k]) || x[j] == x[k])
46.            return false;
47.    }
48.    return true; // 没有冲突，返回true
49.}
50.
51.
52.bool Queen::Backtrack(int t) {
53.    if (t > n) { // 如果已经放置了所有的皇后
54.        cout << "Solution found:" << endl; // 输出找到解决方案的
   消息
55.        for (int i = 1; i <= n; ++i) { // 遍历每一列
56.            for (int j = 1; j <= n; ++j) { // 遍历每一行
57.                cout << ((x[j] == i) ? "Q " : "- "); // 如果该位
   置有皇后则输出 "Q"，否则输出 "-"
58.            }
59.            cout << endl; // 换行
60.        }
61.        return true; // 返回找到解决方案
62.    }
63.    for (int i = 1; i <= n; ++i) { // 遍历当前列的每一行
64.        x[t] = i; // 尝试将皇后放置在当前列的第 i 行
65.        if (Place(t) && Backtrack(t + 1)) // 如果放置合法且成功解
   决剩余的皇后问题
66.            return true; // 返回找到解决方案
67.    }
68.    return false; // 没有找到解决方案
69.}
70.
71.
72.bool Queen::QueensLV(int stopVegas) {
73.    RandomNumber rnd;   // 创建一个随机数生成器对象
74.    int k = 1;          // 当前要放置的列号
75.    int count = 1;      // 有效的放置计数器
76.
77.    // 迭代直到放置到stopVegas 列或者没有有效放置
78.    while ((k <= stopVegas) && (count > 0)) {
79.        count = 0;
80.
```

```
81.         // 尝试将皇后放置在第 k 列的每一行
82.         for (int i = 1; i <= n; ++i) {
83.             x[k] = i;
84.             // 如果不冲突，记录有效放置的位置
85.             if (Place(k))
86.                 y[count++] = i;
87.         }
88.
89.         // 如果有有效放置位置，从中随机选择一个放置皇后
90.         if (count > 0)
91.             x[k++] = y[rnd.Random(count)];
92.     }
93.
94.     // 如果找到了有效的放置方案，调用回溯算法继续搜索剩余的皇后位置
95.     if (count > 0) {
96.         Backtrack(stopVegas + 1);
97.         return true;
98.     }
99.
100.     // 如果没有找到有效的放置方案，返回 false
101.     return false;
102. }
103.
104.
105. bool nQueen(int n) {
106.     Queen X(n); // 创建一个 Queen 对象，传入皇后数量
107.
108.     int stop = (n > 15) ? n - 15 : 8; // 根据皇后数量确定拉斯维
     加斯算法停止的列数
109.
110.     if (X.QueensLV(stop)) { // 使用拉斯维加斯算法放置皇后
111.         for (int i = 1; i <= n; ++i)
112.             cout << X.x[i] << " "; // 输出皇后的列数
113.         cout << endl;
114.         return true; // 返回找到解决方案
115.     }
116.     return false; // 没有找到解决方案
117. }
118.
119.
120. int main() {
121.     nQueen(8);
122.     cout << "Press Enter to exit...";
```

```
    cin.get();
    return 0;
}
```

【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

拉斯维加斯方法是一种基于随机化的算法，通常用于在大量可能性中快速找到解决方案。它通过随机选择来简化复杂的计算过程，并在某些情况下比确定性算法更有效。

举个例子，比如说现在有一把锁，100 把钥匙，只有一把钥匙能够打开锁。那么我们每次随机拿 1 把钥匙进行尝试，打不开就更换 1 把。当我们尝试的次数越多，打开的机会就越大，但在打开之前，那些错的钥匙都是没有用的。这个试钥匙的方法就是拉斯维加斯方法——尽量找到最好的，但不保证能找到。

从这个例子中，我们可以总结得到拉斯维加斯方法具有以下方法特点：

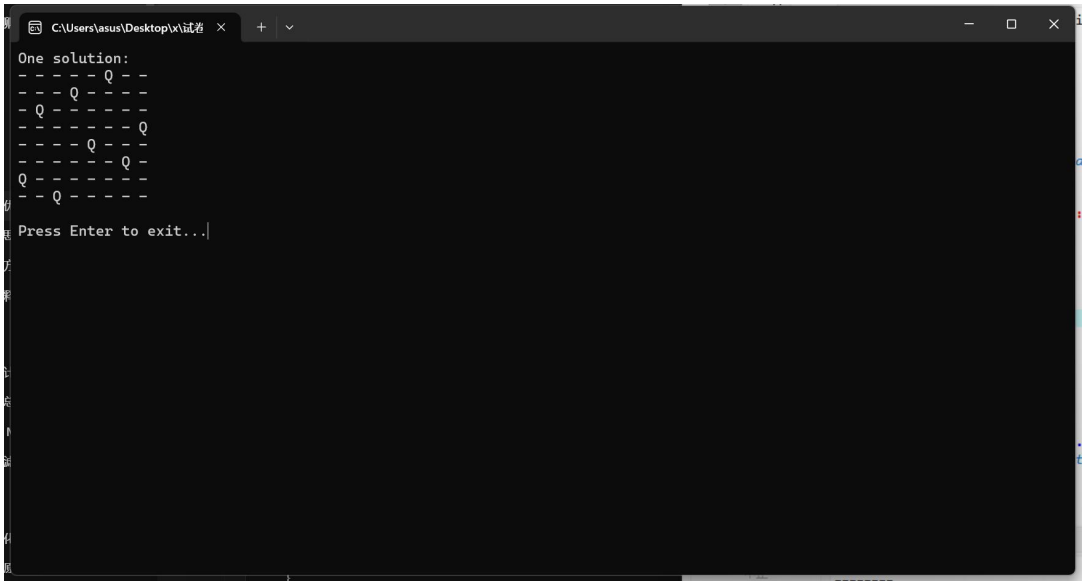
1、快速性：在合适的情况下，拉斯维加斯方法可以比传统的确定性算法更快找到可行的解，因为它不需要遍历所有可能性。

2、概率性：该方法依赖于随机选择，因此同样的问题可能在不同运行中得到不同的结果，甚至可能在某些运行中未能找到解决方案（尽管在多数情况下会找到）。

3、灵活性：结合了随机化和回溯算法的优点，适应不同规模的问题。例如，在 8 皇后问题中，它能有效处理复杂的放置情况。

4、实现简便：通过简单的随机选择和有效性检查就能实现强大的搜索功能，代码相对简洁。

【5、AC 截图和运行测试截图】



【1、实验题目：举例并实现舍伍德方法】

【2、代码实现】

```
#include <iostream>
```

```cpp
#include <vector>
#include <algorithm>

using namespace std;

// 获取中位数
int medianOfMedians(vector<int>& a, int left, int right) {
    if (right - left < 5) {  //找到的中位数作为枢纽元
        sort(a.begin() + left, a.begin() + right + 1);
        return a[left + (right - left) / 2];
    }

    for (int i = left; i <= right; i += 5) { //将数组分成多个组，每组最多五个元素，并将找到的中位数移动至最左侧
        int subRight = min(i + 4, right);
        sort(a.begin() + i, a.begin() + subRight + 1);
        int medianIdx = i + (subRight - i) / 2;
        swap(a[medianIdx], a[left + (i - left) / 5]);
    }

    return medianOfMedians(a, left, left + (right - left) / 5);
    //递归找到中位数的中位数
}

//分区函数
int partition(vector<int>& a, int left, int right, int pivot) {
    for (int i = left; i <= right; ++i) { //找到枢纽元并将其移动到数组的最右侧
        if (a[i] == pivot) {
            swap(a[i], a[right]);
            break;
        }
    }

    int i = left;
    for (int j = left; j < right; ++j) { //对数组进行分区
        if (a[j] < pivot) {
            swap(a[i], a[j]);
            ++i;
        }
    }
    swap(a[i], a[right]);
    return i;  //返回枢纽元的最终位置
```

```
}

int selectK(vector<int>& a, int left, int right, int k) {
    while (true) {
        if (left == right) {
            return a[left];
        }

        int pivot = medianOfMedians(a, left, right); //使用 medianOfMedians 函数找到枢纽元
        int pivotIndex = partition(a, left, right, pivot); //对数组进行分区，并找到枢纽元的位置

        int rank = pivotIndex - left + 1;
        if (rank == k) { //若正好找到
            return a[pivotIndex];
        } else if (rank > k) {  //若枢纽元比 k 大，则往左边找
            right = pivotIndex - 1;
        } else {     //若枢纽元比 k 小，则往右边找
            k = k - rank;
            left = pivotIndex + 1;
        }
    }
}

int main() {
    vector<int> a = {1, 8, 4, 9, 0, 32, 45, 27, 6, 5};
    int k = 5;
    int ans = selectK(a, 0, a.size() - 1, k);
    cout << "The " << k << "-th smallest element is: " << ans << endl;
    return 0;
}
```

【3、遇到的问题及解决方法】

初始定义的算法边界条件未厘清，导致算法陷入死循环，后更新边界，增加 flag 判断符号，成果结束。

【4、算法分析与知识点】

应用：在以上实例中，我们使用舍伍德算法解决了给定一个未排序的数组，要求找出其中第 k 小的元素。一般的方法包括排序后选择第 k 小的元素，但这种方法的时间复杂度为 O(nlogn)。舍伍德方法能够在最坏情况下以线性时间复杂度 O(n) 解决该问题。

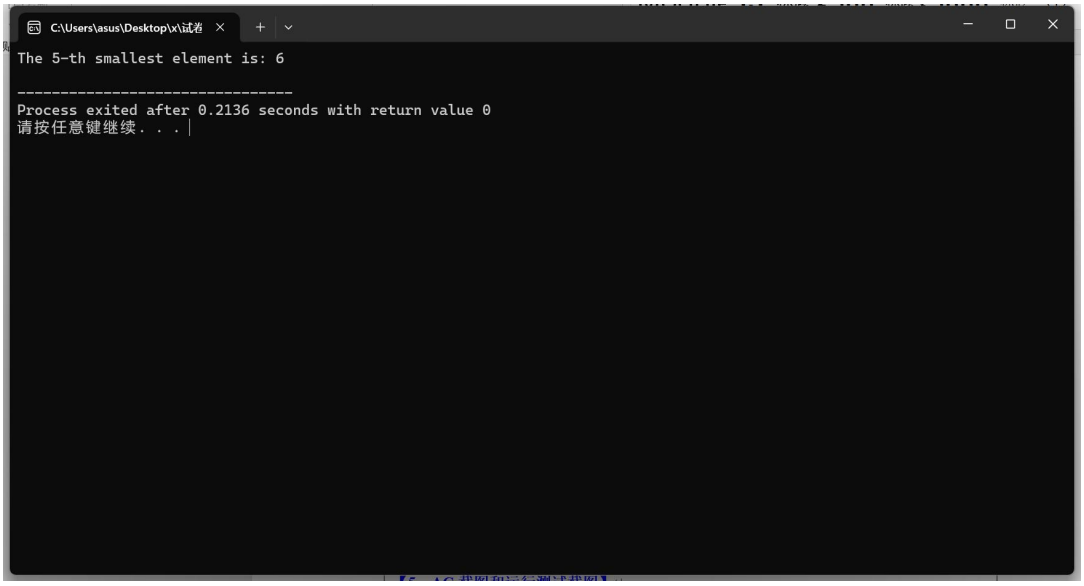
特点：SherWood 算法不是避免了最坏情况的发生，最坏情况还是有可能发生，但是使用舍伍德算法可以消除输入对算法性能的影响。

在上述案例中，舍伍德方法通过分组排序、递归选择中位数的中位数、分区操作以

及调整搜索范围等步骤，有效地实现了在最坏情况下线性时间复杂度 O(n) 找到第 k 小元素。其特点包括在最坏情况下具有线性时间复杂度、稳定性和实用性，使其在解决选择问题时非常高效和可靠。

【5、AC 截图和运行测试截图】

```
int main() {
    vector<int> a = {1, 8, 4, 9, 0, 32, 45, 27, 6, 5};
    int k = 5;
    int ans = selectK(a, 0, a.size() - 1, k);
    cout << "The " << k << "-th smallest element is: " << ans << endl;
    return 0;
}
```



【1、 实验题目：用蒙特卡洛方法，解决 GPS 在隧道中无信号时的导航问题。】

【2、代码实现】

```
#include <iostream>
#include <vector>
#include <random>
#include <cmath>
#include <numeric>  // std::accumulate
#include <algorithm> // std::lower_bound, std::shuffle
#include <cassert>

// 随机数引擎
std::default_random_engine gen((unsigned)time(nullptr));

// 随机采样高斯分布
double sampleGaussian(double mean, double stddev) {
    std::normal_distribution<double> dist(mean, stddev);
    return dist(gen);
}
```

```cpp

// 结构体：表示一个粒子（二维位置 + 朝向 + 权重）
struct Particle {
    double x;
    double y;
    double theta;   // 航向角，单位：弧度
    double weight;  // 归一化权重
};

// 隧道参数（示例：长 1000m，宽 10m，中心线为 x 方向）
constexpr double TUNNEL_LENGTH = 1000.0;   // 隧道长度（m）
constexpr double TUNNEL_WIDTH  = 10.0;     // 隧道宽度（m）
constexpr double TUNNEL_Y_MIN  = -TUNNEL_WIDTH / 2.0;
constexpr double TUNNEL_Y_MAX  =  TUNNEL_WIDTH / 2.0;

// 粒子滤波参数
constexpr int    N_PARTICLES   = 2000;          // 粒子数量
constexpr double SIGMA_ODOM_D  = 0.1;           // 里程计行驶距离噪声 (m)
constexpr double SIGMA_ODOM_TH = 0.0174533;     // 里程计转向噪声 (rad) ~1°
constexpr double SIGMA_LIDAR    = 0.2;          // LiDAR 测距噪声 (m)
constexpr double RESAMPLE_THRESH = N_PARTICLES / 2.0; // 有效粒子数阈值

// 全局粒子容器
std::vector<Particle> particles;

// 初始化粒子：假设车辆在隧道入口 (x=0, y=0, theta=0) 附近，小范围高斯分布
void initializeParticles() {
    std::normal_distribution<double> dist_x(0.0, 1.0);
    std::normal_distribution<double> dist_y(0.0, TUNNEL_WIDTH / 4.0);
    std::normal_distribution<double> dist_th(0.0, SIGMA_ODOM_TH * 5);

    particles.resize(N_PARTICLES);
    for (int i = 0; i < N_PARTICLES; i++) {
        particles[i].x     = dist_x(gen);
        particles[i].y     = dist_y(gen);
        particles[i].theta = dist_th(gen);
```

```
        particles[i].weight = 1.0 / N_PARTICLES;
    }
}

// 运动模型更新：对所有粒子应用里程计增量，并加入噪声
// delta_d：真是测量的行驶距离； delta_th：真实测量的航向变化
void motionUpdate(double delta_d, double delta_th) {
    for (auto &p : particles) {
        // 加噪声
        double noisy_d  = delta_d  + sampleGaussian(0.0, SIGMA_ODOM_D);
        double noisy_th = delta_th + sampleGaussian(0.0, SIGMA_ODOM_TH);

        // 简化 Ackermann 运动模型
        double prev_th = p.theta;
        p.x     += noisy_d * std::cos(prev_th + noisy_th / 2.0);
        p.y     += noisy_d * std::sin(prev_th + noisy_th / 2.0);
        p.theta += noisy_th;

        // 保持 theta 在 [-pi, +pi]
        if (p.theta > M_PI)       p.theta -= 2.0 * M_PI;
        else if (p.theta < -M_PI) p.theta += 2.0 * M_PI;

        // 如果粒子跑出隧道外，则将其权重置零
        if (p.x < 0.0 || p.x > TUNNEL_LENGTH ||
            p.y < TUNNEL_Y_MIN || p.y > TUNNEL_Y_MAX) {
            p.weight = 0.0;
        }
    }
}

// 简化测距模型：假设 LiDAR 只测车辆朝向正交左右两侧到隧道壁的垂直距离。
// lidar_measure[0]：测量车辆左侧到墙面的距离；
// lidar_measure[1]：测量车辆右侧到墙面的距离。
// 返回每个粒子的测距似然乘积（未归一化）
void measurementUpdate(const double lidar_measure[2]) {
    // 首先，所有粒子的权重乘以前一时刻的权重（保持原逻辑）
    // 然后计算新的观测似然
    double weight_sum = 0.0;
```

```
    for (auto &p : particles) {
        if (p.weight == 0.0) continue;  // 失效粒子略过

        // 计算粒子在隧道中对应的左右两侧到壁距离（几何推导）
        // 当粒子航向 theta = 0 时，左侧垂直方向就是 y 正方向
        // 因此，可以简单地认为：
        //   left_dist_hat  = (TUNNEL_WIDTH/2 - p.y) / |cos(theta)|
        //   right_dist_hat = (p.y + TUNNEL_WIDTH/2) / |cos(theta)|
        // 这里忽略了航向角对测距方向的影响，实际情况可用射线投射替换此处逻辑。
        double cos_th = std::abs(std::cos(p.theta));
        if (cos_th < 1e-6) {
            // 当 cos(theta) 过小时，粒子完全侧向朝向隧道壁，简化为极大距离
            p.weight = 0.0;
            continue;
        }
        double left_hat  = ( TUNNEL_Y_MAX - p.y ) / cos_th;  // y = +宽/2 的墙
        double right_hat = ( p.y - TUNNEL_Y_MIN ) / cos_th;  // y = -宽/2 的墙

        // 计算高斯似然
        double e_l = lidar_measure[0] - left_hat;
        double e_r = lidar_measure[1] - right_hat;
        double p_l = std::exp(- e_l * e_l / (2.0 * SIGMA_LIDAR * SIGMA_LIDAR));
        double p_r = std::exp(- e_r * e_r / (2.0 * SIGMA_LIDAR * SIGMA_LIDAR));

        // 更新权重
        p.weight = p.weight * (p_l * p_r);
        weight_sum += p.weight;
    }

    // 归一化权重
    if (weight_sum < 1e-15) {
        // 如果所有粒子权重都接近 0，重新初始化或均匀分布
        std::cerr << "Warning: All particle weights nearly zero. Reinitializing weights uniformly.\n";
        double uniform_w = 1.0 / N_PARTICLES;
```

```
        for (auto &p : particles) {
            p.weight = uniform_w;
        }
    } else {
        for (auto &p : particles) {
            p.weight /= weight_sum;
        }
    }
}

// 计算有效粒子数 N_eff = 1 / sum(w_i^2)
double computeEffectiveSampleSize() {
    double sum_sq = 0.0;
    for (auto &p : particles) {
        sum_sq += p.weight * p.weight;
    }
    if (sum_sq < 1e-15) return 0.0;
    return 1.0 / sum_sq;
}

// 低方差重采样（Low-variance Resampling）
void resampleParticles() {
    std::vector<Particle> new_particles;
    new_particles.reserve(N_PARTICLES);

    // 构造累积权重 CDF
    std::vector<double> cdf(N_PARTICLES);
    cdf[0] = particles[0].weight;
    for (int i = 1; i < N_PARTICLES; i++) {
        cdf[i] = cdf[i - 1] + particles[i].weight;
    }
    assert(std::abs(cdf.back() - 1.0) < 1e-6);  // 权重和应近似为 1

    // 随机初始起始点 u0 ~ Uniform[0, 1/N]
    std::uniform_real_distribution<double> uni_dist(0.0, 1.0 / N_PARTICLES);
    double u0 = uni_dist(gen);

    int idx = 0;
    for (int m = 0; m < N_PARTICLES; m++) {
        double u = u0 + double(m) / N_PARTICLES;
        // 在 cdf 中找到第一个 k 使得 cdf[k] >= u
```

```
        while (u > cdf[idx] && idx < N_PARTICLES - 1) {
            idx++;
        }
        // 复制该 idx 对应的粒子
        new_particles.push_back(particles[idx]);
        new_particles.back().weight = 1.0 / N_PARTICLES;
    }

    // 替换旧粒子集
    particles.swap(new_particles);
}

// 根据加权平均计算当前车体位姿估计
void estimatePose(double &est_x, double &est_y, double &est_theta) {
    est_x = 0.0;
    est_y = 0.0;
    double sin_sum = 0.0;
    double cos_sum = 0.0;

    for (auto &p : particles) {
        est_x    += p.weight * p.x;
        est_y    += p.weight * p.y;
        sin_sum  += p.weight * std::sin(p.theta);
        cos_sum  += p.weight * std::cos(p.theta);
    }
    est_theta = std::atan2(sin_sum, cos_sum);
}

// 模拟获取 LiDAR 左右测距（示例中的真实车辆 y=0，左右距离 = 宽/2 + 噪声）
// 在实际系统中，应由硬件或 API 获取
void getSimulatedLidar(double real_y, double lidar_measure[2]) {
    // 真实测距
    double true_left  = TUNNEL_Y_MAX - real_y;
    double true_right = real_y - TUNNEL_Y_MIN;

    // 加入噪声
    std::normal_distribution<double> dist_l(0.0, SIGMA_LIDAR);
    std::normal_distribution<double> dist_r(0.0, SIGMA_LIDAR);
    lidar_measure[0] = true_left  + dist_l(gen);
    lidar_measure[1] = true_right + dist_r(gen);
```

```cpp
}

int main() {
    // 1. 初始化粒子
    initializeParticles();

    // 假设车辆真实状态（用于模拟测量），初始 (x=0, y=0, theta=0)
    double true_x = 0.0, true_y = 0.0, true_th = 0.0;

    // 仿真循环：假设车辆每步沿隧道中心线向前行驶 1m，共 100 步
    constexpr int STEPS = 100;
    for (int k = 0; k < STEPS; k++) {
        // === 1) 真实车辆运动（用于模拟测量） ===
        double real_delta_d  = 1.0;  // 向前行驶 1m
        double real_delta_th = 0.0;  // 保持直行
        true_x += real_delta_d * std::cos(true_th);
        true_y += real_delta_d * std::sin(true_th);
        true_th += real_delta_th;

        // === 2) 粒子运动预测 ===
        motionUpdate(real_delta_d, real_delta_th);

        // === 3) 模拟 LiDAR 测距 ===
        double lidar_measure[2] = {0.0, 0.0};
        getSimulatedLidar(true_y, lidar_measure);

        // === 4) 测量更新（加权） ===
        measurementUpdate(lidar_measure);

        // === 5) 计算 N_eff 并决定是否重采样 ===
        double N_eff = computeEffectiveSampleSize();
        if (N_eff < RESAMPLE_THRESH) {
            resampleParticles();
        }

        // === 6) 位姿估计 ===
        double est_x, est_y, est_th;
        estimatePose(est_x, est_y, est_th);

        // 输出每步的估计与真实值对比
        std::cout << "Step " << k
                  << " | True: (x=" << true_x << ", y=" << tru
e_y << ", th=" << true_th << ")"
                  << " | Est: (x=" << est_x  << ", y=" << est_
```

```
   y  << ", th=" << est_th << ")"
249.                   << "\n";
250.     }
251.
252.     return 0;
253. }
```

【3、遇到的问题及解决方法】

CDF 构造的断言未考虑浮点误差在重采样部分使用了 assert(std::abs(cdf.back() - 1.0) < 1e-6) 来判断权重归一化是否成功。但由于浮点误差，即使合理归一化也可能略超出阈值。

【4、算法分析与知识点】

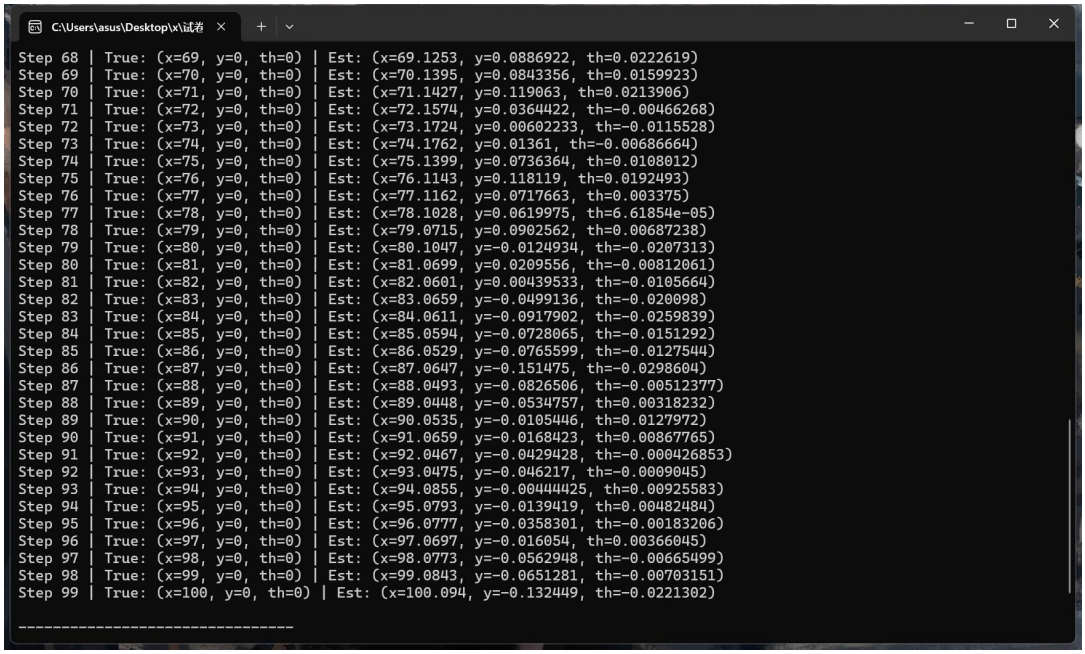
在隧道中导航时，由于 GPS 信号不可用，蒙特卡洛方法（即粒子滤波）通过构建大量粒子来近似表示车辆可能的状态分布。每个粒子代表一个位置和朝向估计，初始化时在隧道可能区域内均匀分布。随着车辆移动，所有粒子根据运动模型进行状态预测，并加入噪声以模拟真实运动中的误差。这一过程帮助系统维持对车辆位置的不确定性建模，即使没有 GPS 也可持续推断状态。

随后，系统通过传感器（如左右壁距传感器）获取实际观测数据，计算每个粒子与观测之间的匹配程度，并据此调整粒子权重。这些权重反映了每个粒子“代表真实位置”的可能性。经过观测更新后，通过重采样步骤保留高权重粒子并丢弃低权重粒子，从而提升定位精度。最终通过加权平均得到车辆当前位置与朝向的估计。该方法无需依赖 GPS 信号，适用于隧道、地下等封闭场景，具备鲁棒性强、适应性高的优点。

基于蒙特卡洛方法的隧道导航算法，其时间和空间复杂度主要由粒子数 NNN 决定。每轮迭代中，所有粒子都需经历运动更新、观测更新、重采样和状态估计等步骤。由于每个步骤对每个粒子仅进行一次处理，因此整体的时间复杂度为 O(N)O(N)O(N)。若算法运行 TTT 个时间步，则总时间复杂度为 O(NT)O(NT)O(NT)，即线性增长，具有较好的实时性。

空间复杂度方面，每个粒子需存储其位置、朝向以及权重信息，所需空间为常数级别 O(1)O(1)O(1)，总粒子数为 NNN，因此整体空间复杂度为 O(N)O(N)O(N)。这意味着只要粒子数控制在合理范围内，该算法在资源受限的嵌入式导航设备中也具备良好的适应性和可扩展性。

【5、AC 截图和运行测试截图】



【第二部分/本实验无需扩展】